

Repository Summary: Eternal Torment

Franklin Edward Diaz

pale-shadow/eternal-torment

May 5, 2026

Overview

Purpose: A high-entropy sandbox repository dedicated to low-level coding practice, infrastructure-as-code (IaC) modeling, and continuous learning.

Context: Part of the `pale-shadow` organization, this repository serves as a functional testing ground for CTF tools, assembly experiments, and the recently integrated `ml-gnn` machine learning project. It bridges the gap between raw hardware interaction and cloud-scale infrastructure analysis.

Recent Updates: Project ml-gnn

Recent activity focuses on the intersection of Graph Neural Networks and Cloud Security. The primary objective is the automated extraction and analysis of Terraform-based infrastructure digraphs.

- **Infrastructure Graph Collection:** A Python pipeline (`src/collection/`) that extracts relationship data from Terraform plans.
- **Data Lifecycle Management:** Integration of DVC (Data Version Control) for dataset tracking and `.metadata.json` consistency.
- **GCP Integration:** Automated exfiltration of DOT, PNG, and Terraform state files to the `backend-datastore` storage bucket.

Setup & Environment Configuration

The project now supports multi-environment builds via GNU Autotools and containerized runtimes.

Docker Workflow

```
docker build -t gnn-collection .
```

Native Environment (Bash)

```
./bootstrap.sh  
./configure  
make python
```

```
export BASE_DIR=$(pwd)
source ${BASE_DIR}/_build/bin/activate
export PYTHONPATH=${BASE_DIR}
```

Native Environment (Fish Shell)

```
./bootstrap.sh
./configure
make python
source /home/franklin/workspace/ml-gnn/_build/bin/activate.fish
set --export PYTHONPATH /home/franklin/workspace/ml-gnn
```

Execution

With the environment active, the collection engine is executed from within a Terraform workspace directory:

```
cd /path/to/terraform/workspace
python3 ${BASE_DIR}/src/collection/collection.py
```

Computational Performance Metrics

Numerical processing for GNN embeddings and large-scale graph analysis is offloaded to the `lab.bitmasher.net` NVIDIA Jetson cluster. Throughput is modeled as:

$$P_{total} = \sum_{i=1}^n (N_{cores, i} \times f_i \times FLOPS_{cycle})$$

Where $n=4$ nodes, $N_{cores, i}$ represents CUDA cores, and f_i represents clock frequency.

Technical Stack

- **Languages:** Assembly (AArch64), C, Python, Shell (Bash/Fish/Ksh).
- **Build Systems:** GNU Autotools, Make.
- **Cloud/IaC:** Terraform, Google Cloud Platform (GKE, Storage).
- **Security:** Bandit (SAST), Tox (Automation).